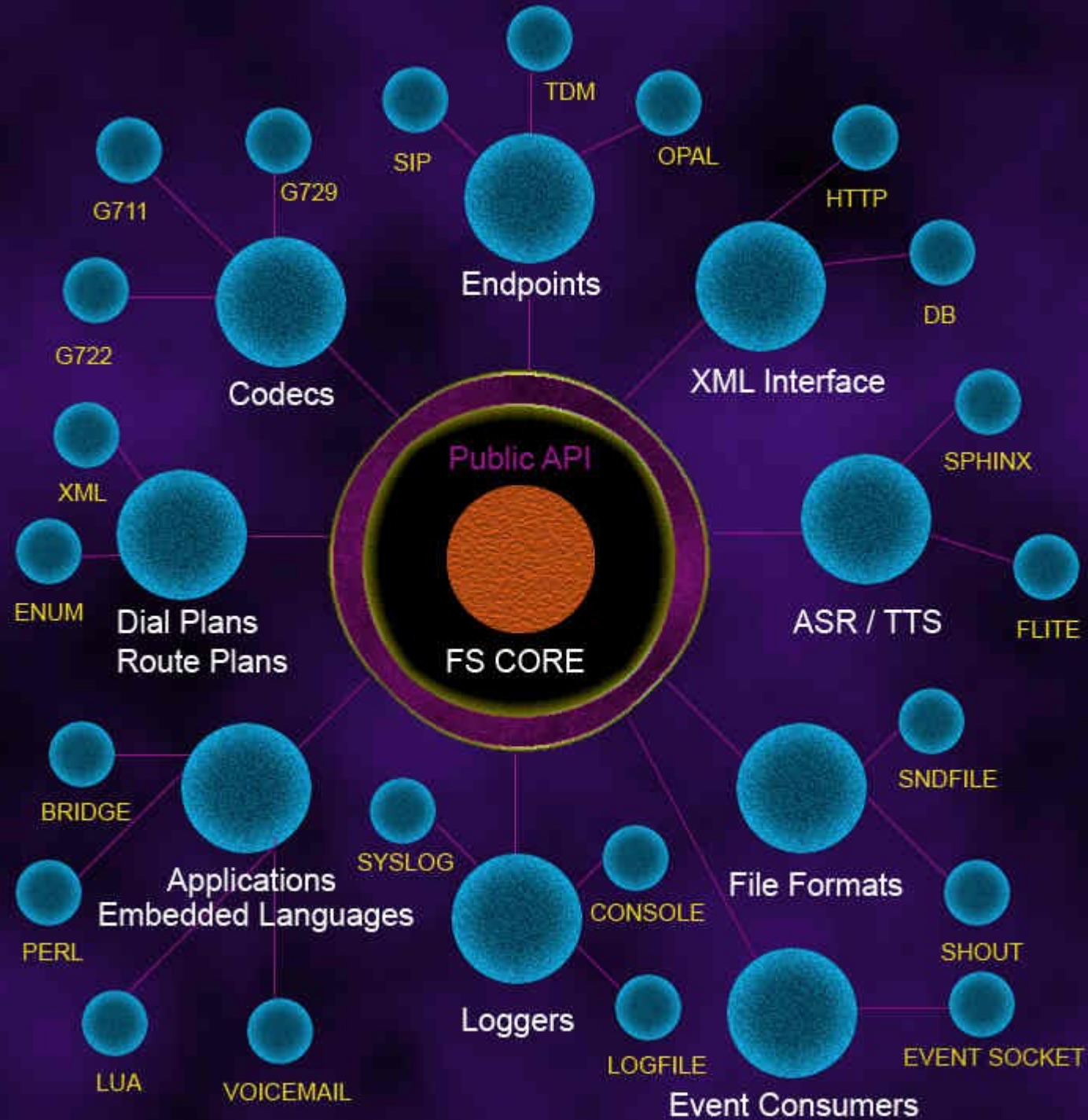


How to build a FreeSWITCH event_handler module

Norm Brandinger
n.brandinger@gmail.com



What are FreeSWITCH Events?

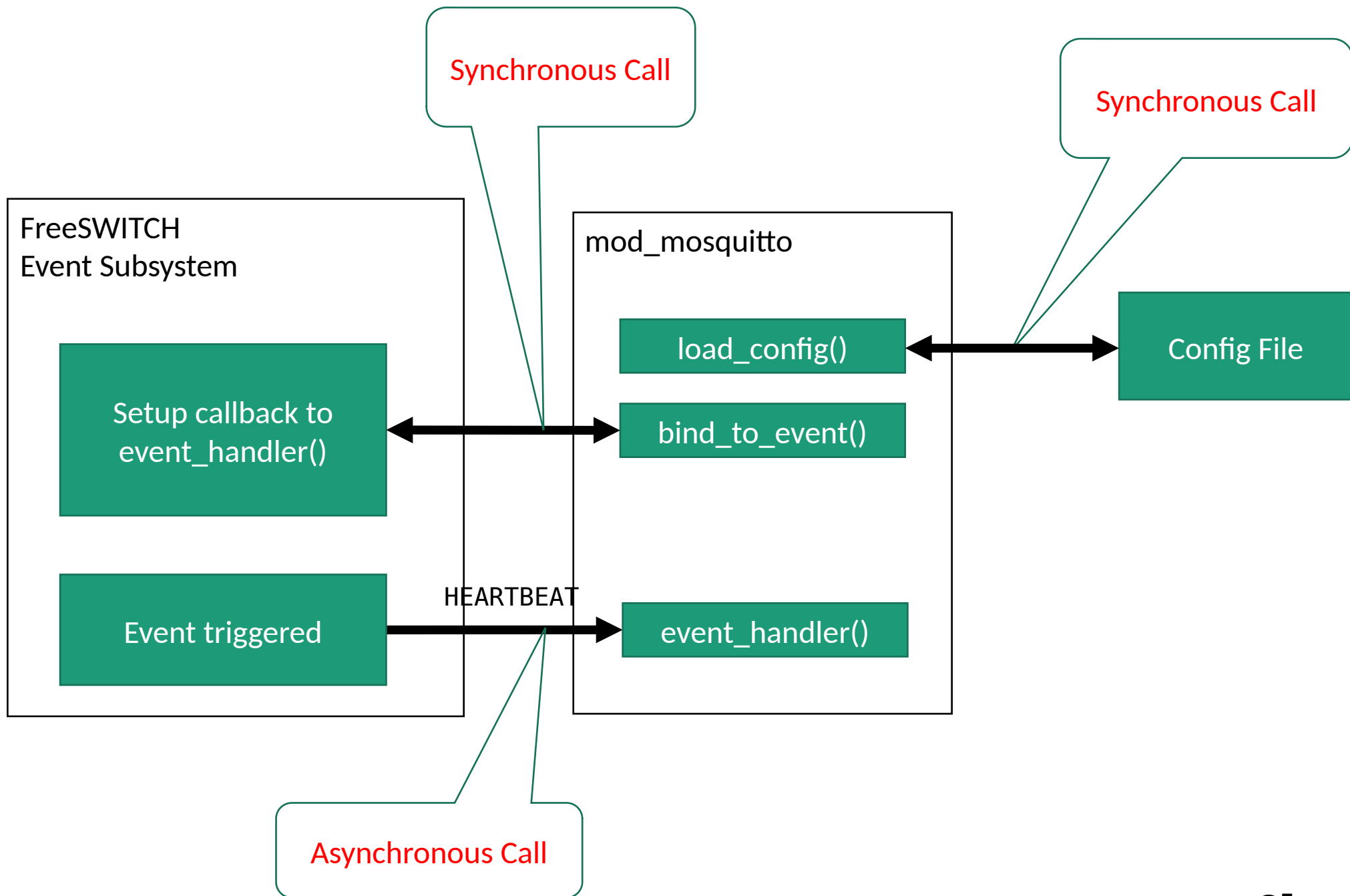
- Triggered asynchronously

HEARTBEAT

CHANNEL_CREATE

BACKGROUND_JOB

- Pass real-time information among different subsystems
- Internal and external modules can subscribe to events



What are FreeSWITCH Modules?

- Expand the core functions
- Focus on performing a stated purpose
- Compiled Library of functions

Most are C/C++

Most are configured via XML

FreeSWITCH Modules by category

61	Applications
5	Automatic Speech Recognition (ASR) / Text to Speech (TTS)
22	Codecs
2	Databases
3	Dial Plans
1	Directories
16	Endpoints
19	Event Handlers
13	Formats
8	Languages
5	Loggers
18	Say
2	Timers
6	XML Interfaces

181	

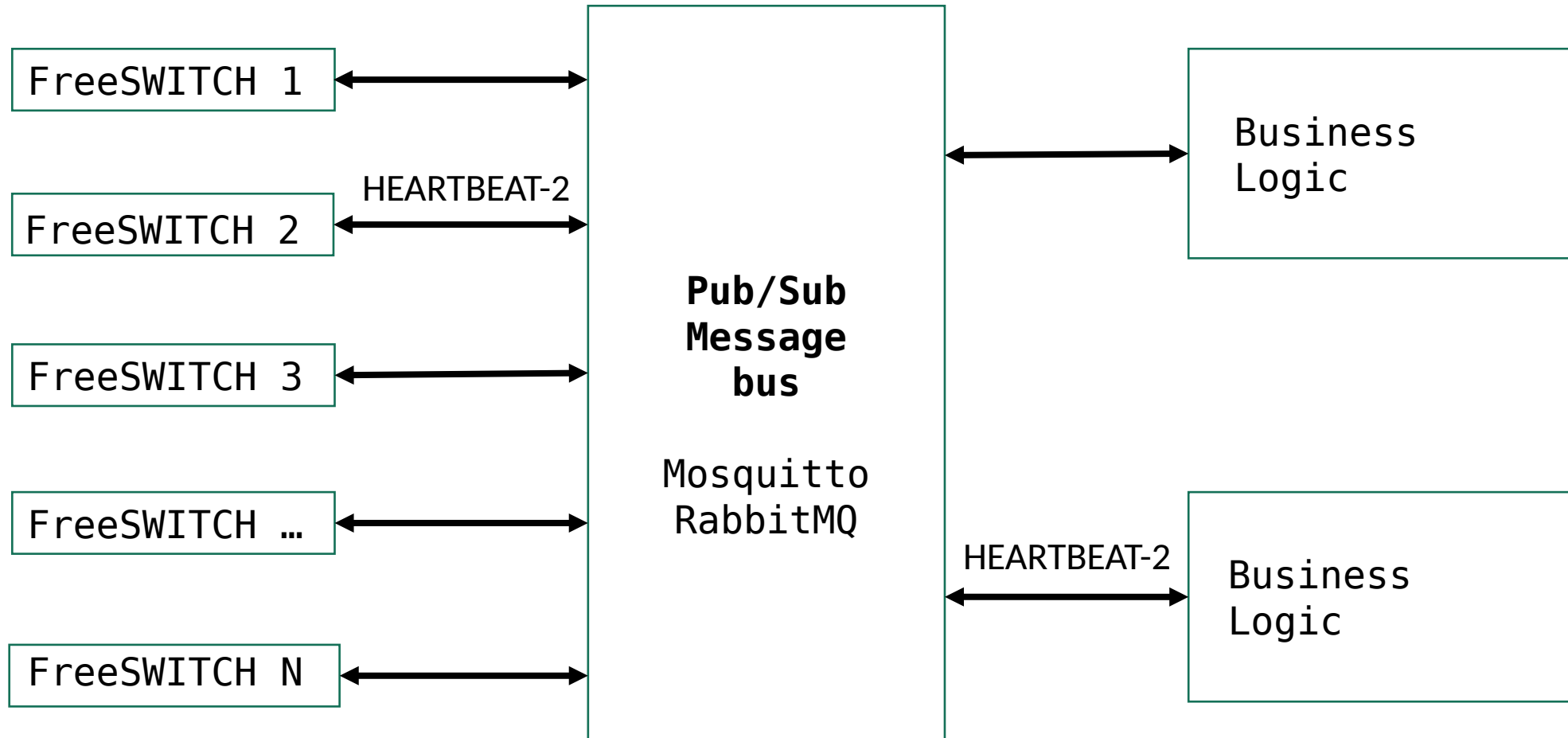
Why create another FreeSWITCH module?

`mod_mosquitto`

Implements an MQTT interface to/from FreeSWITCH
to a publish/subscribe message broker

- MQTT implements the publish-subscribe messaging pattern
- Mosquitto is a message broker that implements MQTT
- RabbitMQ is a message broker that supports MQTT via a core plugin

Publish / Subscribe Messaging Pattern Decoupled Architecture



Architecture Design Recap

- FreeSWITCH was designed to adapt to changing technologies
- FreeSWITCH has a robust Event Subsystem
- Modules are a mechanism to extend the capabilities from FreeSWITCH and can take advantage of the Event Subsystem
- mod_mosquitto uses the Event Subsystem and the external Mosquitto library to add pub/sub messaging using MQTT support

Technology Level set

- Download, compile, install FreeSWITCH from source

Prerequisite software

- Autotools
- Familiarity with C/C++

`SWITCH_*`

Useful utility definitions: `switch_utils.h`

```
#define switch_safe_free(it) if (it) {free(it);it=NULL;}
```

Where are modules located?

- freeswitch/src/mod/{category}
- Out of tree source code
 - freeswitch/src/mod/outoftree/mod_mosquitto
 - freeswitch/src/mod/outoftree/mod_example
- freeswitch/src/modules.conf:
 - mod_mosquitto|https://github.com/freeswitch/mod_mosquitto.git -b master
 - mod_example|https://github.com/NormB/mod_example.git -b master

Minimal Module

```
#include <switch.h>

SWITCH_MODULE_LOAD_FUNCTION(mod_example_load);
SWITCH_MODULE_SHUTDOWN_FUNCTION(mod_example_shutdown);
SWITCH_MODULE_RUNTIME_FUNCTION(mod_example_runtime);

//#define SWITCH_MODULE_DEFINITION_EX(name, load, shutdown, runtime, flags)
//static const char modname[] = #name ;
SWITCH_MODULE_DEFINITION(mod_example, mod_example_load, mod_example_shutdown, mod_example_runtime);

//switch_status_t mod_example_load (switch_loadable_module_interface_t **module_interface, switch_memory_pool_t *pool);
SWITCH_MODULE_LOAD_FUNCTION(mod_example_load) {
    *module_interface = switch_loadable_module_create_module_interface(pool, modname);
    return SWITCH_STATUS_SUCCESS;
}
SWITCH_MODULE_RUNTIME_FUNCTION(mod_example_runtime) {
    return SWITCH_STATUS_SUCCESS;
}
SWITCH_MODULE_SHUTDOWN_FUNCTION(mod_example_shutdown) {
    return SWITCH_STATUS_SUCCESS;
}
```

Example Makefile.am

```
AUTOMAKE_OPTIONS = foreign subdir-objects
```

```
MODNAME=mod_example
```

```
mod_LTLIBRARIES = mod_example.la
```

```
mod_example_la_SOURCES = mod_example.c
```

```
mod_example_la_CFLAGS = $(FREESWITCH_CFLAGS)
```

```
mod_example_la_LDFLAGS = -avoid-version -module -no-undefined -shared $(FREESWITCH_LIBS)
```

mod_example.h

```
#define log(severity, ...) switch_log_printf(SWITCH_CHANNEL_LOG, severity, __VA_ARGS__);

#define CONFIG_FILE "example.conf"

typedef struct event_s {
    char *name;                // readable event name used for hash table searching
    switch_event_types_t event_type; // event id (enum)
    switch_event_node_t *node;    // structure returned by the event subsystem
} event_t;

typedef struct globals_s {
    switch_memory_pool_t *pool; // memory pool for our use, provided by to the module
    switch_api_interface_t api_interface; // needed for api (cli) access
    switch_hash_t *events; // hash table used to hold the events we are listening for
    switch_mutex_t *mutex; // control access to the hash table
} globals_t;

extern globals_t globals;
```

mod_example_load

```
SWITCH_MODULE_LOAD_FUNCTION(mod_example_load) {
    memset(&globals, 0, sizeof(globals));
    globals.pool = pool;

    switch_mutex_init(&globals.mutex, SWITCH_MUTEX_DEFAULT, globals.pool);
    switch_core_hash_init(&globals.events);

    /* create a loadable module interface structure named with modname */
    /* the module interface defines the different interfaces that this module has defined */
    *module_interface = switch_loadable_module_create_module_interface(pool, modname);

    /* create an api interface for the console - command line interface (cli) */
    add_cli_api(module_interface, &globals.api_interface);

    load_config(CONFIG_FILE);

    return SWITCH_STATUS_SUCCESS;
}
```

example.conf.xml

```
<!-- freeswitch/conf/autoload_configs/example.conf.xml -->  
<configuration name="example.conf" description="mod_example">  
  <events>  
    <param name="event" value="HEARTBEAT"/>  
    <param name="event" value="CHANNEL_CREATE"/>  
  </events>  
</configuration>
```

load_config

```
switch_status_t load_config(const char *cf) {
    switch_xml_t cfg, xml, events, param;

    xml = switch_xml_open_cfg(cf, &cfg, NULL);

    if ((events = switch_xml_child(cfg, "events"))) {
        for (param = switch_xml_child(events, "param"); param; param = param->next) {
            char *var = NULL;
            char *val = NULL;
            var = (char *)switch_xml_attr_soft(param, "name");
            val = (char *)switch_xml_attr_soft(param, "value");
            if (!strncasecmp(var, "event", 5) && !zstr(val)) {
                event_t *event = NULL;
                event = (event_t *)switch_core_alloc(globals.pool, sizeof(event_t));
                event->name = switch_core_strdup(globals.pool, val);
                switch_name_event(event->name, &event->event_type);
                switch_event_bind_removable("example", event->event_type, SWITCH_EVENT_SUBCLASS_ANY, event_handler,
                NULL, &event->node);
                switch_core_hash_insert_locked(globals.events, event->name, event, globals.mutex);
                log(SWITCH_LOG_INFO, "Added new event: %s\n", event->name);
            }
        }
    }
    switch_xml_free(xml);
    return SWITCH_STATUS_SUCCESS;
}
```

event_handler

```
void event_handler(switch_event_t *event) {  
    char *buf;  
    //  
    // create a string from the JSON event  
    //  
    switch_event_serialize_json(event, &buf);  
  
    log(SWITCH_LOG_DEBUG, "%s\n", buf);  
    switch_safe_free(buf);  
}
```

add_cli_api

```
// From the mod_example_load() routine
// create an api interface for the module command line interface (cli)
// add_cli_api(module_interface, &globals.api_interface);

#define EXAMPLE_DESC "Example API"
#define EXAMPLE_SYNTAX "example status"

void add_cli_api(switch_loadable_module_interface_t **module_interface, switch_api_interface_t
*api_interface)
{
    //
    // exec_api_cmd is called whenever "example" is entered on the console
    //
    SWITCH_ADD_API(api_interface, "example", EXAMPLE_DESC, exec_api_cmd, EXAMPLE_SYNTAX);

    switch_console_set_complete("add example status");
}
```

exec_api_cmd

```
SWITCH_STANDARD_API(exec_api_cmd)
{
    //
    // console command: "example status"
    //
    if (!strncasecmp(argv[0], "status", 6)) {
        switch_mutex_lock(globals.mutex);
        for (switch_hash_index_t *events_hi = switch_core_hash_first(globals.events);
events_hi; events_hi = switch_core_hash_next(&events_hi)) {
            event_t *event = NULL;
            void *val;
            switch_core_hash_this(events_hi, NULL, NULL, &val);
            event = (event_t *)val;
            stream->write_function(stream, "Event: %s\n", event->name);
        }
        switch_mutex_unlock(globals.mutex);
    }
    //
    //
}
```

mod_example_shutdown

```
SWITCH_MODULE_SHUTDOWN_FUNCTION(mod_example_shutdown) {
    remove_cli_api();

    switch_mutex_lock(globals.mutex);
    for (switch_hash_index_t *events_hi = switch_core_hash_first(globals.events); events_hi; events_hi
= switch_core_hash_next(&events_hi)) {
        event_t *event = NULL;
        void *val;
        switch_core_hash_this(events_hi, NULL, NULL, &val);
        event = (event_t *)val;
        switch_event_unbind(&event->node);
    }
    switch_mutex_unlock(globals.mutex);

    switch_core_hash_destroy(&globals.events);
    switch_event_unbind_callback(event_handler);
    switch_mutex_destroy(globals.mutex);
    return SWITCH_STATUS_SUCCESS;
}
```

fs_cli

```
freeswitch@> load mod_example
```

```
[INFO] mod_example.c:124 Added new event: HEARTBEAT
```

```
[INFO] mod_example.c:124 Added new event: CHANNEL_CREATE
```

```
[CONSOLE] switch_loadable_module.c:1803 Successfully Loaded [mod_example]
```

```
[NOTICE] switch_loadable_module.c:412 Adding API Function 'example'
```

```
[DEBUG] mod_example.c:101 {"Event-Name":"HEARTBEAT","Core-UUID":"c231dd83-acd8-46d3-b9f8-591246ef7ea7","FreeSWITCH-Hostname":"","FreeSWITCH-Switchname":"","FreeSWITCH-IPv4":".","FreeSWITCH-IPv6":"","1","Event-Date-Local":"2020-08-02 13:29:44","Event-Date-GMT":"Sun, 02 Aug 2020 17:29:44 GMT","Event-Date-Timestamp":"1596389384034308","Event-Calling-File":"switch_core.c","Event-Calling-Function":"send_heartbeat","Event-Calling-Line-Number":"81","Event-Sequence":"7531","Event-Info":"System Ready","Up-Time":"0 years, 0 days, 15 hours, 26 minutes, 59 seconds, 638 milliseconds, 515 microseconds","FreeSWITCH-Version":"1.10.4-dev+git~20200720T221847Z~8a5850ff26~64bit","Uptime-msec":"55619638","Session-Count":"0","Max-Sessions":"1000","Session-Per-Sec":"30","Session-Per-Sec-Last":"0","Session-Per-Sec-Max":"0","Session-Per-Sec-FiveMin":"0","Session-Since-Startup":"0","Session-Peak-Max":"0","Session-Peak-FiveMin":"0","Idle-CPU":"99.933333"}
```

```
freeswitch@> example status
```

```
Event: HEARTBEAT
```

```
Event: CHANNEL_CREATE
```

```
freeswitch@> unload mod_example
```

```
[NOTICE] switch_loadable_module.c:1318 Deleting API Function 'example'
```

```
[DEBUG] switch_loadable_module.c:1320 Write lock interface 'example' to wait for existing references.
```

```
[CONSOLE] switch_loadable_module.c:2396 Stopping: mod_example
```

```
[DEBUG] switch_event.c:2187 Event Binding deleted for example:HEARTBEAT
```

```
[DEBUG] switch_event.c:2187 Event Binding deleted for example:CHANNEL_CREATE
```

```
[CONSOLE] switch_loadable_module.c:2416 mod_example unloaded.
```

Summary

- A working `event_handler` module
- Common programming constructs
- Code to read and parse configuration data
- Subscribing to the Event Subsystem
- Setting up and parsing a console command
- Asynchronous execution safeguards using a mutex
- Hash table operations

Thank You

Example module presented here: `github.com/NormB/mod_example`

Pub/Sub implementation over MQTT: `github.com/freeswitch/mod_mosquitto`

Creating Modules <https://freeswitch.org/confluence/display/FREESWITCH/Creating+New+Modules>

Events <https://freeswitch.org/confluence/display/FREESWITCH/Events>

Norm Brandinger
`n.brandinger@gmail.com`

ClueCon 2020